Ministry of Communications and Information Technology

NTI
National Telecommunication Institute

Digital Design using FPGA

Prepared by: Mohammed Salah

What is HDL?

- HDL standing for Hardware Description Language
- It is not a programming language
- You must design before coding (think hardware)
- Two main languages are being used nowadays Verilog & VHDL.

By: Mohammed Salah

Verilog constructs

- The Verilog files have the extension .v
- Verilog is case sensitive
- Verilog line of code terminates with semicolon ‘;’
- Comments like in C by “//” for single line and “/*…..*/” for multiple lines
- The code here executes in parallel unlike the normal programing languages

By: Mohammed Salah

Numeric constants

- For a single wire, the data value that can appear on it is the following:

Representation	Value
“Low” logic zero	0
“High” logic one	1
High impedance	Z
unknown	X

- ‘X’ is used by the simulators only to represent the signals that are not initialized by a known value

By: Mohammed Salah

Numeric constants

- For a couple of wires “BUS”, we can assign data to the whole bus without assigning wire by wire:

Syntax	Symbol	Numbering system
Bus = 100 or Bus = ‘d100	d	Decimal (default)
Bus = ‘b0110_0100	b	Binary
Bus = ‘o144	o	Octal
BUS = ‘h64	h	Hexadecimal

By: Mohammed Salah

Numeric constants

- General numbering representation

```
[sign][size]'[base][value]
```

Number	Stored value	Comment
5'b11010	11010	
5'b11_010	11010	_ignored
5'o32	11010	
5'h1a	11010	
5'd26	11010	
5'b0	00000	0 extended
5'b1	00001	0 extended
5'bz	zzzzz	z extended
5'bx	xxxxx	x extended
5'bx01	xxx01	x extended
-5'b00001	11111	2's complement of 00001

By: Mohammed Salah

OPERATORS

- Bitwise operators
  - Performs bit by bit evaluation between two operands
  - 2'b01 & 2'b11 = {0&1,1&1}
  - ~(3'b011) = 3'b100

Bitwise

NOT	~a
AND	A & b
OR	A | B
XOR	A ^ B
XNOR	A ~^ B A ^~ B

By: Mohammed Salah

OPERATORS

- Logical operators
  - Returns one-bit 1 or 0 (true or false)
  - !(2'b01) = 0
  - "==" tests logical equality for '1' and '0' only all other will result in 'X'
  - "===" tests 4-state logical equality (1,0,Z,X)

Logical

NOT	!A
AND	A && b
OR	A || B
If X or Z in bits returns X	A == B A != B
Return 0 or 1 only based on bit-by-bit comparison	A === B A !== B

By: Mohammed Salah

Ministry of Communications and Information Technology

National Telecommunication Institute

OPERATORS

- reduction operators
  - Evaluate bit-by-bit of a vector
  - &(4'b0111) = {0&1&1&1} = 0
- Arithmetic operators
  - Used to perform arithmetic operations
- Shift operators
  - Shift the first operand by the number specified by the second operand

Reduction

AND	&A
NAND	~&A
OR	|A
NOR	~|A
XOR	^A

Arithmetic

addition	+
subtraction	-
multiplication	*
division	/
modulus	%

Arithmetic shift <<< , >>>	**Logical shift << , >>**
3'b110 >>> 1 = 3'b111	3'b110 >> 1 = 3'b011

By: Mohammed Salah

OPERATORS

- Relational operators
  - Compares 2 operands and returns 1 or 0
- Concatenation and replication operators
  - {2'b00,2'b11,2'b00,2'b11} = 8'b00_11_00_11
  - {4{2'b11},8'b0} = 16'b1111_1111_0000_0000
  - Concatenation and replication have very useful applications, and they are very fast and easy to use
- Conditional operator
  - Condition ? True : False

Relational

Less than	<
Less than or equal	<=
Greater than	>
Greater than or equal	>=
Equal to	==
Not equal to	!=

By: Mohammed Salah

Start coding with Verilog

Verilog Module

- Basic building block in Verilog
- Each module has directional ports (input , output , inout) used to communicate with the module
- A module can be an element or a collection of lower-level design blocks
- A module declared with keyword module
- A corresponding keyword endmodule must appear at the end of the module
- Ports are declared as wires by default

By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Nets**
  - Represent physical connection between hardware component. supply0, supply1, tri/wire, tri0, tri1, triand/wand, trior/wor, trireg
- **Variables**
  - Represent abstract storage storage in behavioral modeling. integer, real, reg, time, realtime
- **Parameters**
  - Run-time constants

    localparam, parameter, specparam

By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Nets**
  - Represent physical connection between hardware component. supply0, supply1, tri/wire, tri0, tri1, triand/wand, trior/wor, trireg

```
module net_example (
    input  a,
    input  b,
    output out_and,
    output out_or
);

    wire internal_node;

    assign internal_node = a & b;
    assign out_and = internal_node;
    assign out_or = a | b;
endmodule
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Nets** **wire**
  - Wire represents a physical wire that is used inside a circuit or module
  - Wire doesn't store its value and so must be continuously driven
  - Bus is an array of wires moving together in parallel to transfer data
  - Note that we can type the width of the bus in the square brackets like this ex. [0:3] but here the MSB will be [0] and LSB will be [3] which is little bit confusing, so we used to write it in this format [MSB:LSB]

```
wire a,b,c;              //three 1-bit wires
wire [31:0] data;        //a 32-bit bus
wire [7:0] b1,b2,b3,b4; //four 8-bit busses
wire [W-1:0] in;         //parameterized bus
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Nets** **wire**
  - It describes a simple physical connection between two structural elements.
  - Any assignments using assign statement must assign to a wire variable.
  - It may take be on of this 4 values : - 0 (Logic 0) - 1 (Logic 1)
    - X (Unknown) - Z (High impudence)
  - For example: wire [15:0] data; // declaration of 16-bit signal of type wire.

```
wire a,b,c;              //three 1-bit wires
wire [31:0] data;        //a 32-bit bus
wire [7:0] b1,b2,b3,b4; //four 8-bit busses
wire [W-1:0] in;         //parameterized bus
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Nets** **Continuous assignment**
  - Continuous assignment are used to describe a behavior equivalent to combinational logic
  - Keyword assign is used for the assignment of wires
  - Continuous assignments are continuously evaluated so when ever the RHS changes, the LFS is updated (typical combinational logic)

By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Variables**
  - Represent abstract storage storage in behavioral modeling. integer, real, reg, time, realtime

```
module variable_example (
    input clk,
    output reg [3:0] counter
    always @(posedge clk) begin
        counter <= counter + 1;
    end
endmodule
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Variables** **reg**
  - It is a variable which may or mayn't describe a hardware register.
  - Any assignments in an always block must assign to a reg variable.
  - Used for applying stimulus in testbench.
  - It may take be on of this 4 values : - 0 (Logic 0) - 1(Logic 1)
    - X (Unknown) - Z (High impedance)
  - For example: reg [15:0] data; // declaration of 16-bit signal of type reg.

By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Variables** **Procedural blocks**

```
always @* begin
    if (sel)
        y = b;      // blocking assignment for combinational
    else
        y = a;
end
```

```
// Stimulus & reset sequencing
initial begin
    // Initialize
    {a, b, sel} = 3'b000;
    rst_n = 0;

    // Release reset
    #12 rst_n = 1;

    // Drive inputs
    #10 a = 1;
    #10 sel = 1;   // y should follow b now
    #10 b = 1;
    #10 sel = 0;   // y should follow a now
    #20;

    $finish;
end
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Variables** **Procedural blocks** **1) always block**

```
always @(sensitivity list)
  begin
    statements;
  end
```

Note: begin and end are only needed when more than one statement inside always block.

- *In* always block the statements are executed in sequence(procedurally).
- The code inside always block is executed when one of the sensitivity list changes.
- All always blocks are executed in parallel.
- Used to describe both combinational and sequential logic.

By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Variables** **Procedural blocks** **2) Initial block**

  - In initial block the statements are executed in sequence(procedurally).
  - Initial block is being used to describe the stimulus or test patterns and test benches for the design
  - The initial block is executed once at time zero.
  - Initial blocks are not synthesizable.
  - # is used to suspend the execution of the initial block for a given delay.

```
initial
  begin
    some statements
  end
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Parameters**
  - Run-time constants

    localparam, parameter, specparam

```
module parameter_example #(
    parameter WIDTH = 8
)(
    input [WIDTH-1:0] data_in,
    output [WIDTH-1:0] data_out
);


    assign data_out = data_in;
endmodule
```


By: Mohammed Salah

Data Types

Verilog provides three *groups* of value objects and different types in each group:

- **Parameters**
  - Compile-time constant inside a module
  - Makes designs **configurable & reusable**
  - Commonly used for **bus widths, sizes, delays**
  - Can be **overritten at instantiation**

```
module parameter_example #(
    parameter WIDTH = 8
)(
    input [WIDTH-1:0] data_in,
    output [WIDTH-1:0] data_out
);


    assign data_out = data_in;
endmodule
```


By: Mohammed Salah

Coding styles

Gate-level modeling

- The module is implemented using logic gates and the interconnection between them
- Outputs of the synthesis tool coded in this style

Structural modeling

- It instantiate predefined modules by the designer and connect them together to build a larger circuit

Behavioral modeling

- Most important modeling style
- Required for complex and sequential designs
- Behavior is described in C-like code

By: Mohammed Salah

Coding styles

Structural

```
module Mux4x1(in1,in2,in3,in4,sel,out);

    input in1,in2,in3,in4;
    input [1:0] sel;
    output reg out;

    wire w1,w2;

    MUX2x1 m0 (.A(in1),.B(in2),.sel(sel[0]),.out(w1));
    MUX2x1 m1 (.A(in3),.B(in4),.sel(sel[0]),.out(w2));
    MUX2x1 m2 (.A(w1),.B(w2),.sel(sel[1]),.out(out));

endmodule
```

Gate level

```
module Mux2x1(A,B,sel,out);

    input A,B,sel;
    output out;

    wire w1,w2,sel_bar;

    not(sel_bar,sel);
    and(w1,A,sel);
    and(w2,B,sel_bar);
    or(w1,w2);

endmodule
```

Behavioral

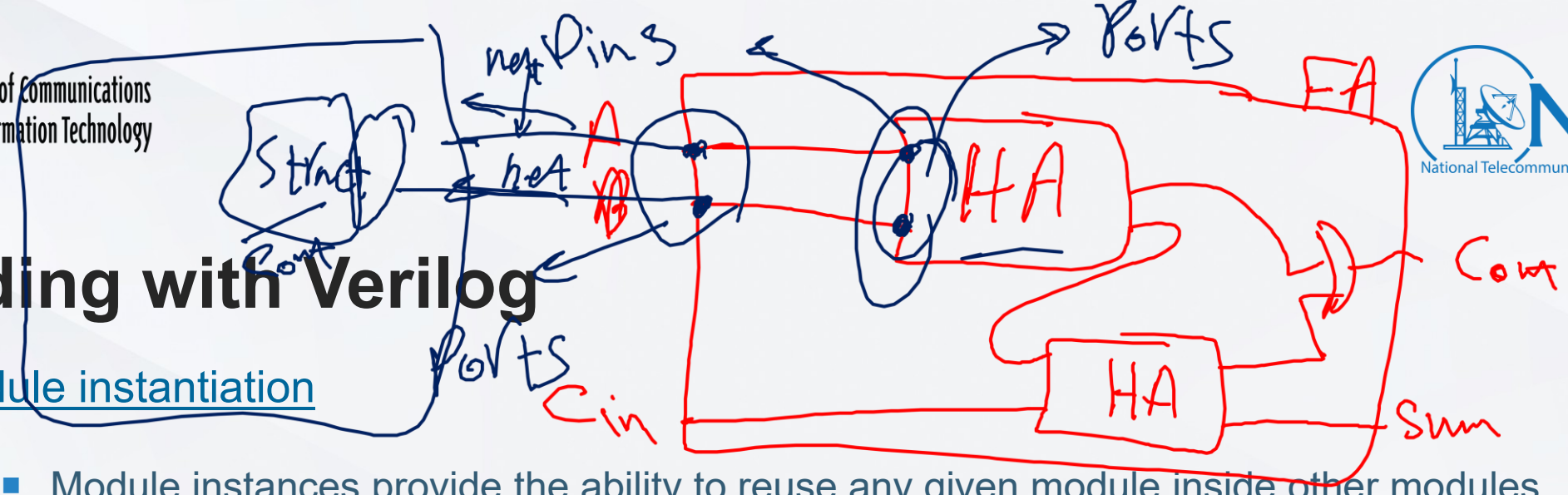
```
module Mux2x1(A,B,sel,out);

    input A,B,sel;
    output reg out;

    always@(*)begin
        case(sel)
            1'b0: out = B;
            1'b1: out = A;
            default out = 1'b1;
        endcase
    end

endmodule
```


By: Mohammed Salah



Coding with Verilog

Module instantiation

- Module instances provide the ability to reuse any given module inside other modules
- There are two ways for port mapping (by name , by order)

```
1  module MUX4X1(A,B,C,D,Sel,Out);
2
3      input A,B,C,D;
4      input [1:0] Sel;
5      output Out;
6
7      wire m0,m1;
8
9      MUX2X1 M0 (.A(A),.B(B),.Sel(Sel[0]),.Out(m0));
10     MUX2X1 M1 (.A(C),.B(D),.Sel(Sel[0]),.Out(m1));
11     MUX2X1 M3 (.A(m0),.B(m1),.Sel(Sel[1]),.Out(Out));
12
13 endmodule
```

Module name

Instance name

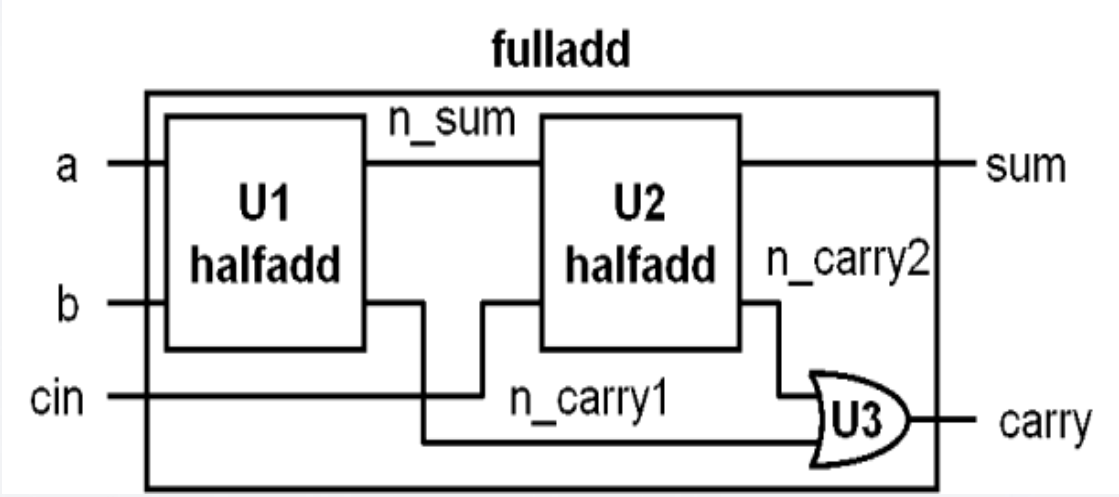
Port connection list

```
1  module MUX2X1(A,B,Sel,Out);
2
3      input A,B,Sel;
4      output Out;
5
6      assign Out = Sel?B:A;
7
8  endmodule
```


By: Mohammed Salah

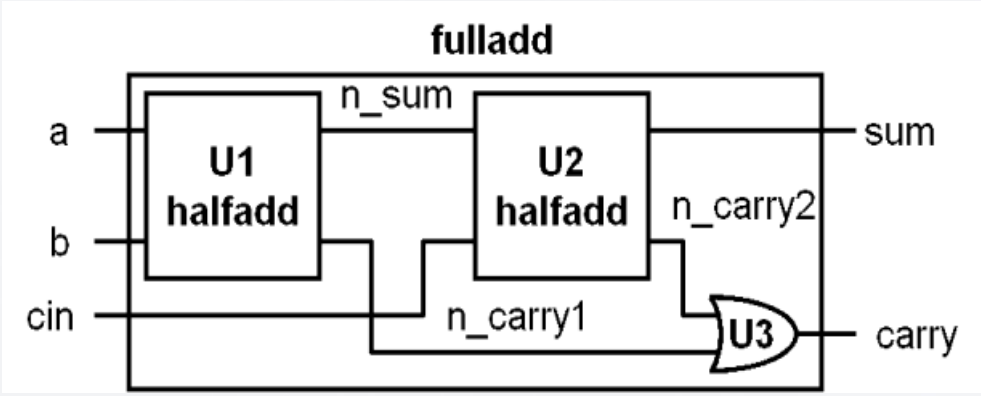
Data Types

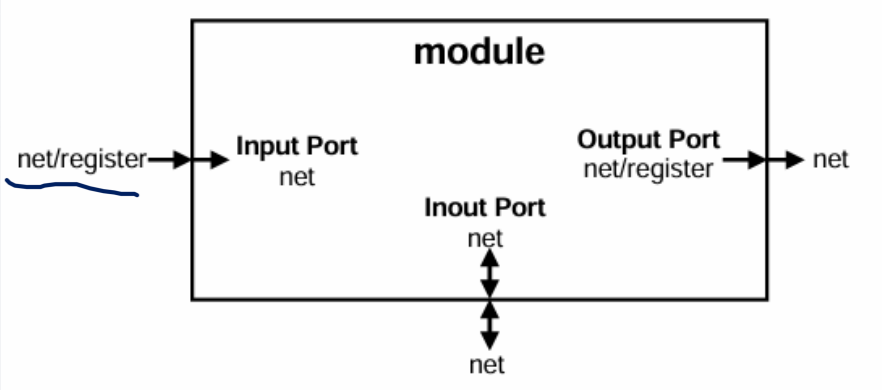
Identify all variable(reg) and net(wire) signals



By: Mohammed Salah

Data Types

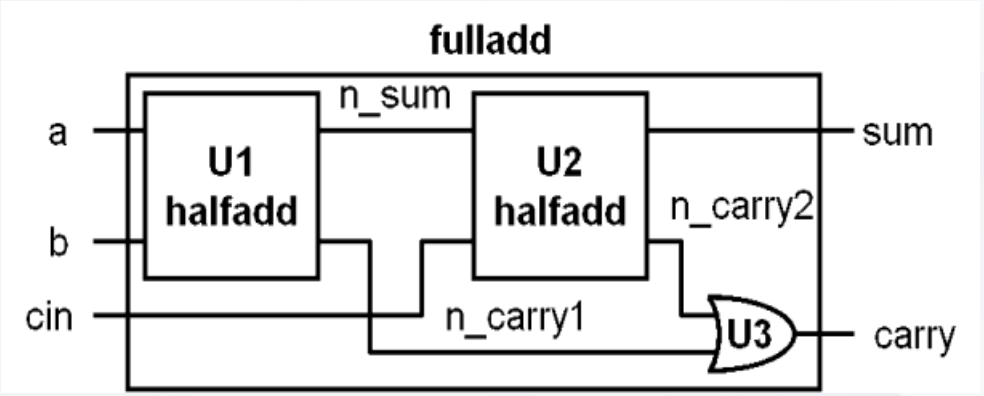




By: Mohammed Salah

Creating Hierarchy

- Declare local and variables.
- Instantiate module(s).
  - Connect instance ports to local nets and variables.
- A port of an instance of a module can be connected using:
  - Named port connection.
  - Ordered port connection.



Ports are implicitly nets if not declared as a variable

```
module fulladd (input a, b, cin, output sum, carry);
  wire n_sum, n_carry1, n_carry2;
  halfadd U1(.a(a), .b(b), .sum(n_sum), .carry(n_carry1));
  halfadd U2(.a(n_sum), .b(cin), .sum(sum), .carry(n_carry2));
  or      U3(carry, n_carry2, n_carry1);
endmodule
```

Built-in OR primitive

Connecting hierarchy-Named port connection

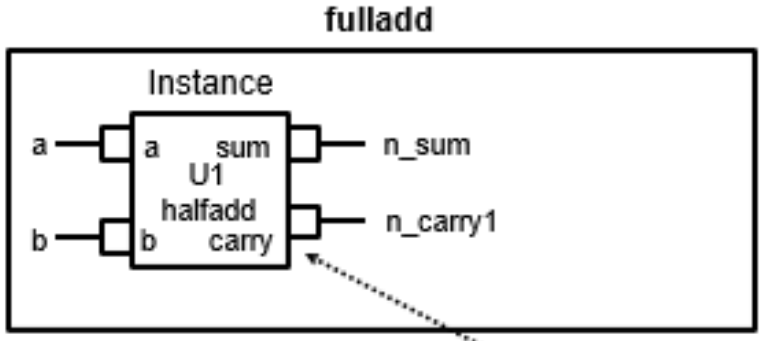
Connecting hierarchy-ordered port connection

By: Mohammed Salah

Connecting Hierarchy – Named Port Connection

Map local port, net or variable to instance port by name – explicitly specify the instance port name.

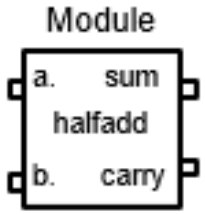
With this syntax you are less likely to make a mistake!




```
module fulladd (input a, b, cin, output sum, carry);
  wire n_sum, n_carry1, n_carry2;
  halfadd U1(.a(a), .b(b), .sum(n_sum), .carry(n_carry1));
  halfadd U2(.a(n_sum), .b(cin), .sum(sum), .carry(n_carry2));
  or      U3(carry, n_carry2, n_carry1);
endmodule
```

Wire *n_carry1* of module *fulladd* mapped to output carry of instance *U1* of module *halfadd*

```
module halfadd (a, b, sum, carry);
input  a, b;
output sum, carry;
  assign sum   = a ^ b;
  assign carry = a & b;
endmodule
```

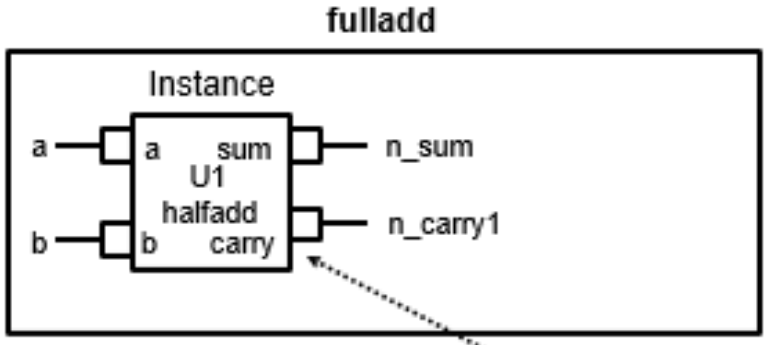


By: Mohammed Salah

Connecting Hierarchy – Ordered Port Connection

Map local port, net or variable to instance port by position – in the order the ports are declared.

With this syntax you can very easily make a mistake!




```
module fulladd (input a, b, cin, output sum, carry);
  wire n_sum, n_carry1, n_carry2;
  halfadd U1(a, b, n_sum, n_carry1);
  halfadd U2(n_sum, cin, sum, n_carry2);
  or      U3(carry, n_carry2, n_carry1);
endmodule
```

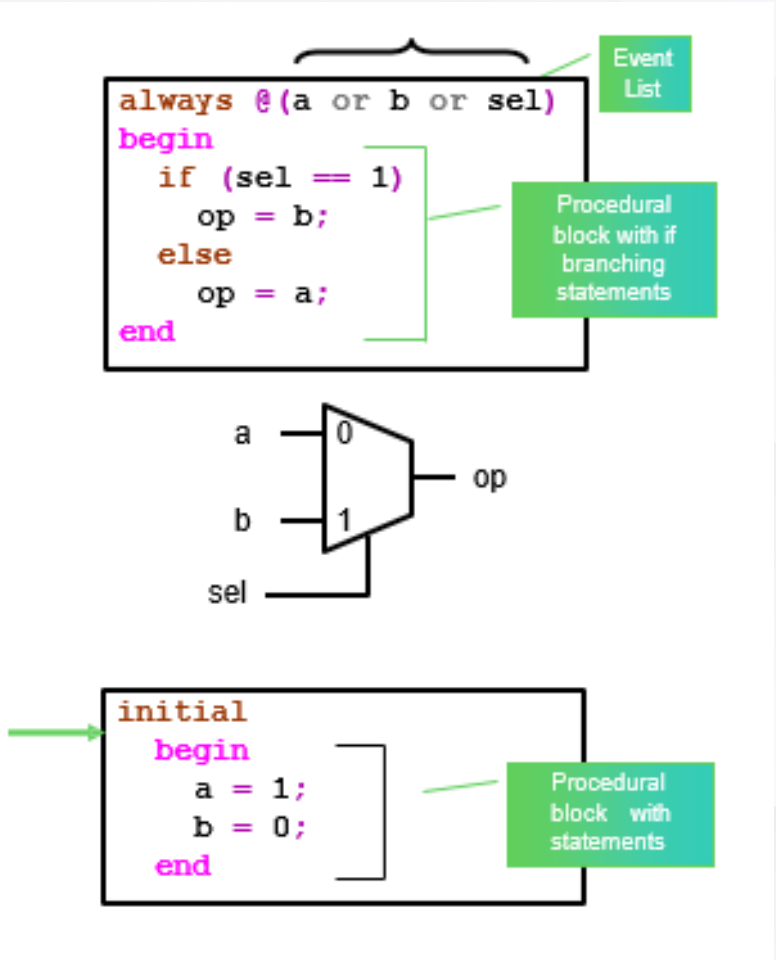
Wire *n_carry1* of module *fulladd* mapped to output *carry* of instance *U1* of module *halfadd*

```
module halfadd (a, b, sum, carry);
input  a, b;
output sum, carry;
  assign sum   = a ^ b;
  assign carry = a & b;
endmodule
```

By: Mohammed Salah

Describing Module Behavior with Procedural Blocks

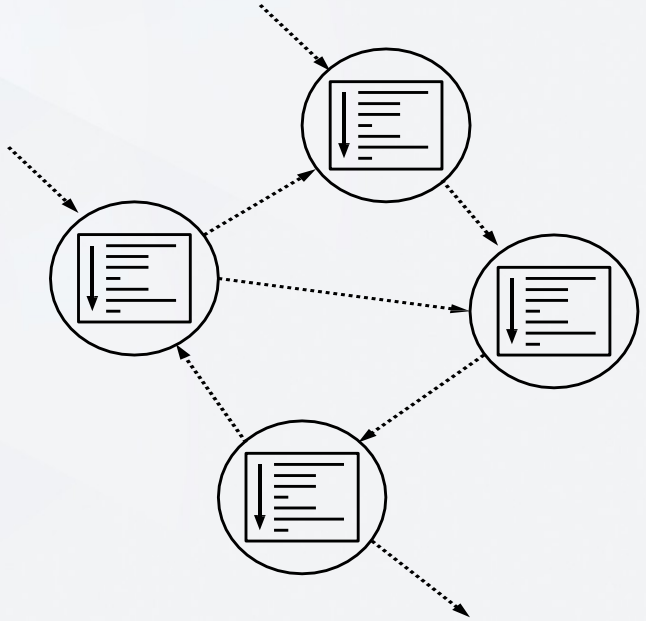
- Procedural Block starts with the keyword:
  - **always**
    - Synthesizable construct
    - Executes at start of simulation
    - Execution blocks and unblocks in accordance with timing controls
    - *When at end, loops back to beginning*
  - **initial**
    - Non-synthesizable or Testbench construct
    - Executes at start of simulation
    - Execution blocks and unblocks in accordance with timing controls
    - When at end, *terminates*



By: Mohammed Salah

Describing Module Behavior with Procedural Blocks

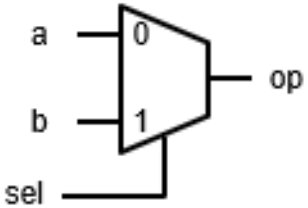
- Multiple statements in procedural blocks are enclosed between *begin* and *end* keywords.
- Multiple procedural blocks interact concurrently



```
always @(a or b or sel)
begin
  if (sel == 1)
    op = b;
  else
    op = a;
end
```

Event List

Procedural block with if branching statements




```
initial
  begin
    a = 1;
    b = 0;
  end
```

Procedural block with statements

By: Mohammed Salah

Synchronizing Module Behaviors

- Use the @ event control.
- Execution blocks until an event in the event expression occurs.
- An event is any transition of the specified nets and variables.
- Verilog-2001 and above added the comma and wildcard operators.
  - The wildcard operator adds all “signals” that go into the block and into any functions called from the block.

Parentheses are optional for event expressions consisting of a single token.

1995 – use *or* operator

```
always @(a or b or sel)
  if (sel == 1)
    op = b;
  else
    op = a;
```

2001 and above – can use , operator

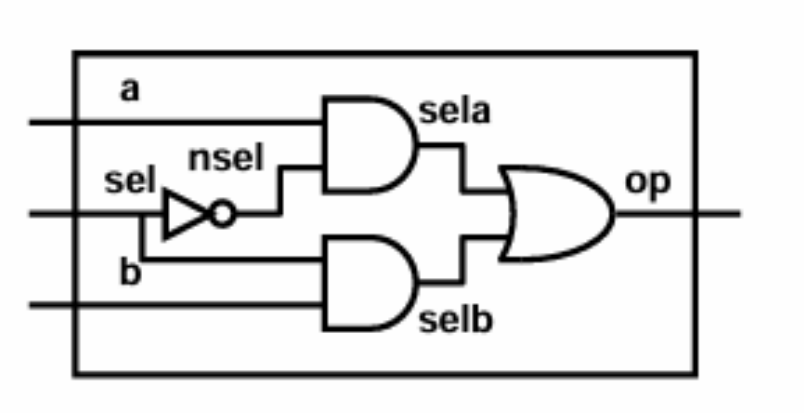
```
always @(a, b, sel)
  if (sel == 1)
    op = b;
  else
    op = a;
```

2001 and above – can use * wildcard for combinatorial logic inputs

```
always @*
  if (sel == 1)
    op = b;
  else
    op = a;
```

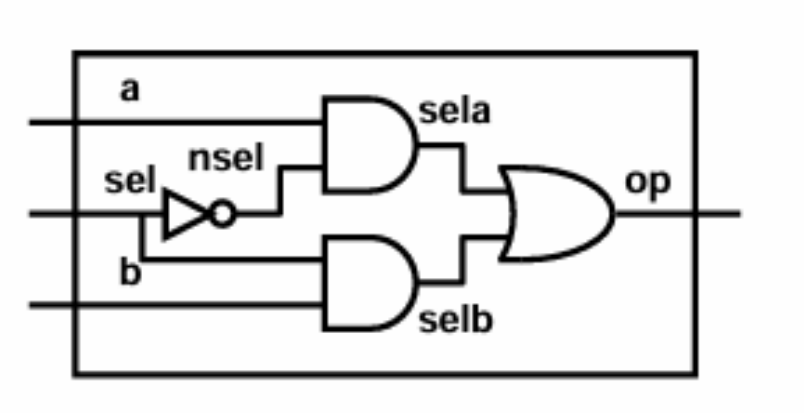

By: Mohammed Salah

Merging Net Declaration and Assignment



By: Mohammed Salah

Merging Net Declaration and Assignment




```
module mux (a, b, sel, op);
  input  sel, b, a;
  output op;
  wire nsel, sela, selb;
  assign nsel = ~sel;
  assign selb =  sel & b;
  assign sela = nsel & a;
  assign op   = sela | selb;
endmodule
```

Bitwise NOT

Bitwise OR

```
module mux (a, b, sel, op);
  input  sel, b, a;
  output op;
  wire nsel = ~sel;
  wire selb =  sel & b;
  wire sela = nsel & a;
  assign op = sela | selb;
endmodule
```


By: Mohammed Salah

Compiler directive

```
module mux (
input a, b, sel,
output reg op
);
  assign op = sel? b : a;
endmodule
```

```
module mux (
    input a, b, sel,
    output op
);
    always @(a, b, sel)
    if (sel == 1)
        op = b;
    else
        op = a;
endmodule
```


By: Mohammed Salah

Compiler directive

```
module halfadd (
 input a, b, output sum, carry );
  // a & b are wire by default
  // drive by continuous assign
  assign svm   = a ^ b;
  assign carry = a & b;
endmodule
```

Undeclared identifier defaults to wire.

By: Mohammed Salah

Compiler directive

```
`default_nettype none
module halfadd (
 input a, b, output sum, carry );
  // a,b,sum,carry are implicit wires
  assign svm   = a ^ b;
  assign carry = a & b;
endmodule
```

Compilation warning on a, b, sum and carry

ERROR: Undeclared identifier

```
module halfadd (
 input a, b, output sum, carry );
  // a & b are wire by default
  // drive by continuous assign
  assign svm   = a ^ b;
  assign carry = a & b;
endmodule
```

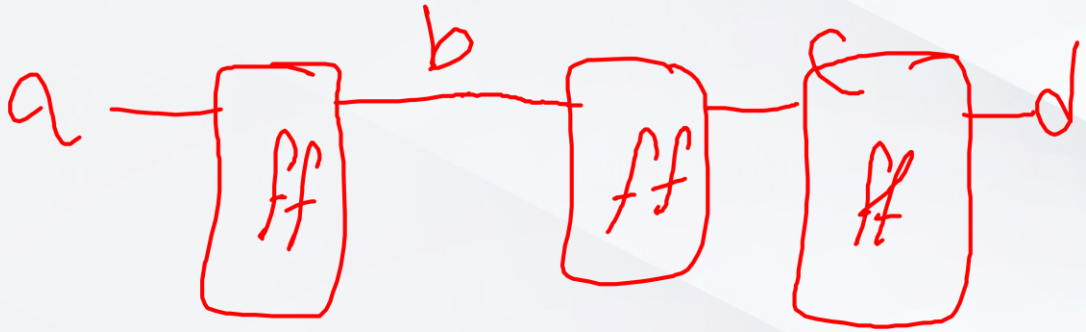
Undeclared identifier defaults to wire.

By: Mohammed Salah

BA Vs NBA

```
always @(posedge clk)
  begin
    b = a;
    c = b;
    d = c;
  end
```

```
always @(posedge clk)
  begin
    d = c;
    c = b;
    b = a;
  end
```

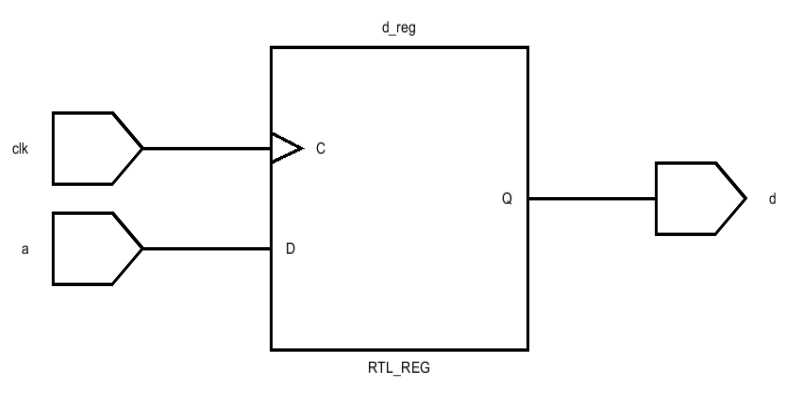


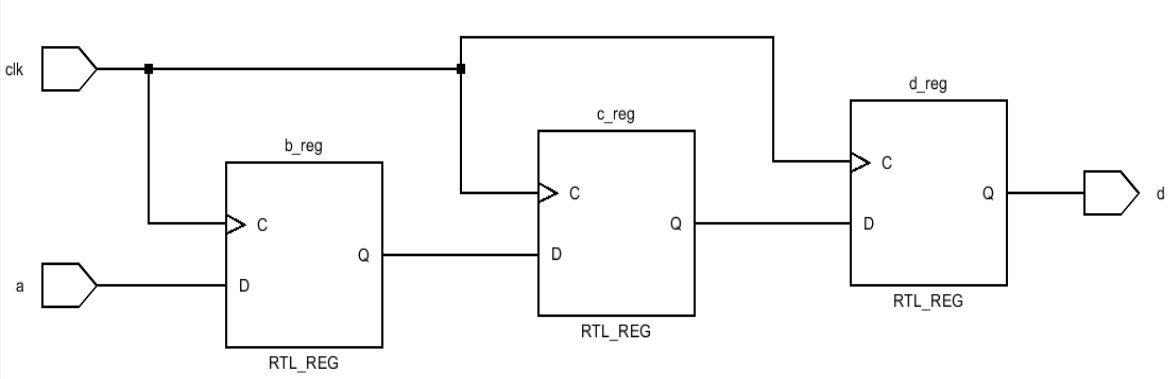
By: Mohammed Salah

BA Vs NBA

```
always @(posedge clk)
  begin
    b = a;
    c = b;
    d = c;
  end
```

```
always @(posedge clk)
  begin
    d = c;
    c = b;
    b = a;
  end
```



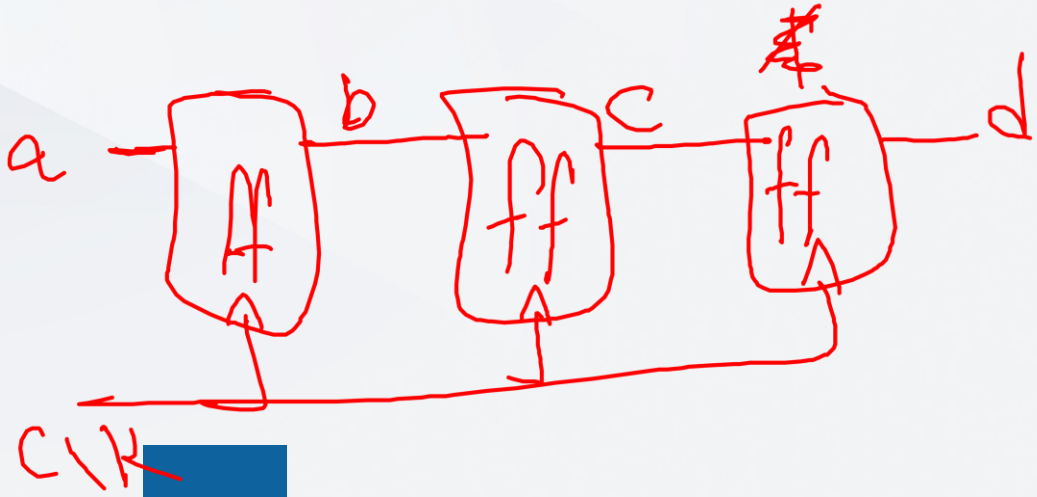


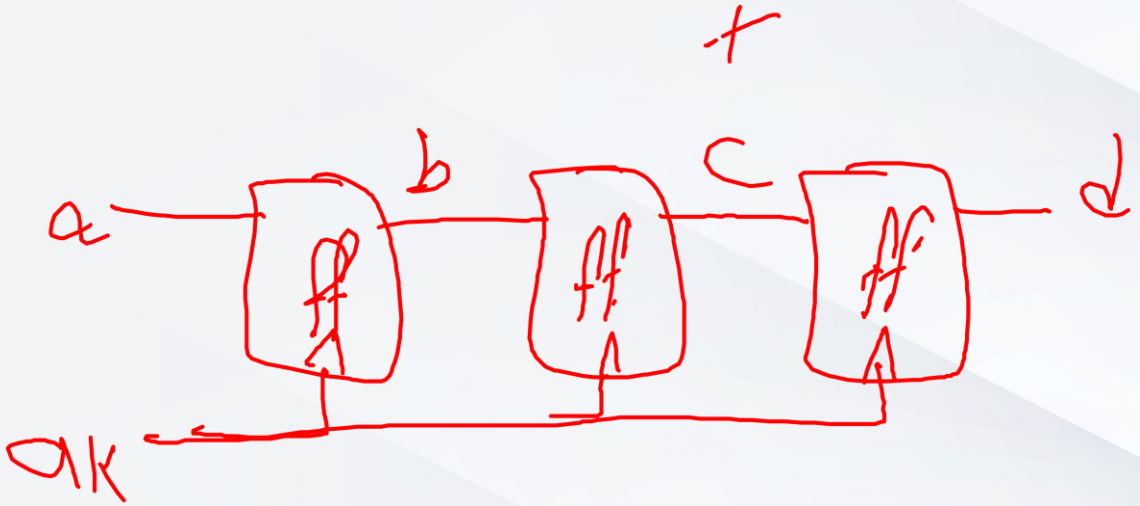
By: Mohammed Salah

BA Vs NBA

```
always @(posedge clk)
  begin
    b <=a;
    c <=b;
    d <=c;
  end
```

```
always @(posedge clk)
  begin
    d <=c;
    c <=b;
    b <=a;
  end
```

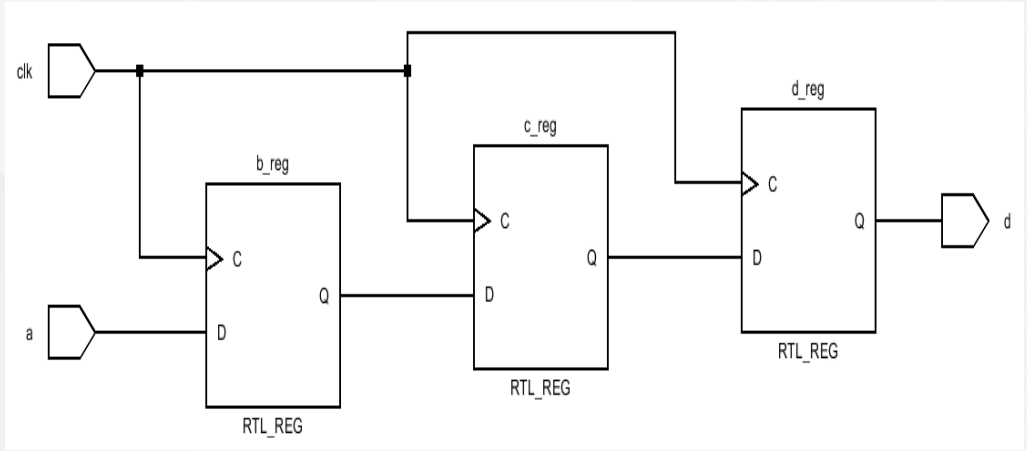


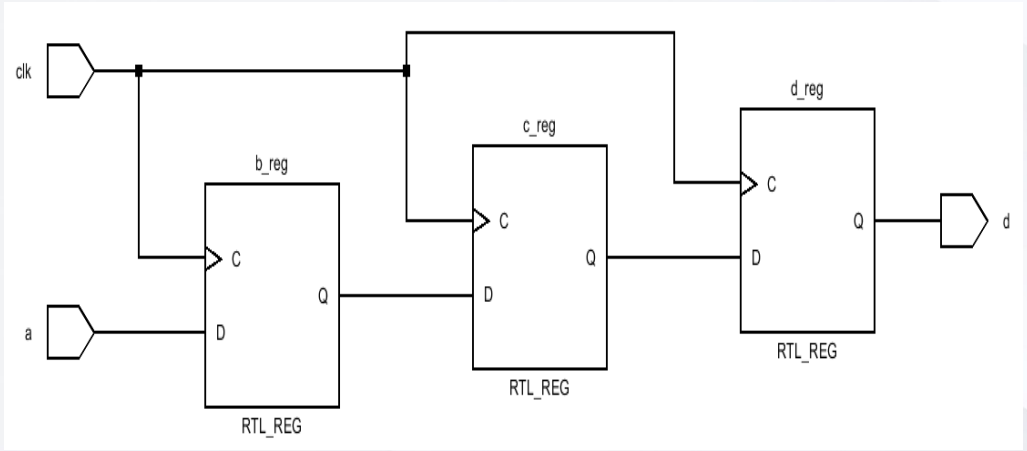


BA Vs NBA

```
always @(posedge clk)
  begin
    b <=a;
    c <=b;
    d <=c;
  end
```

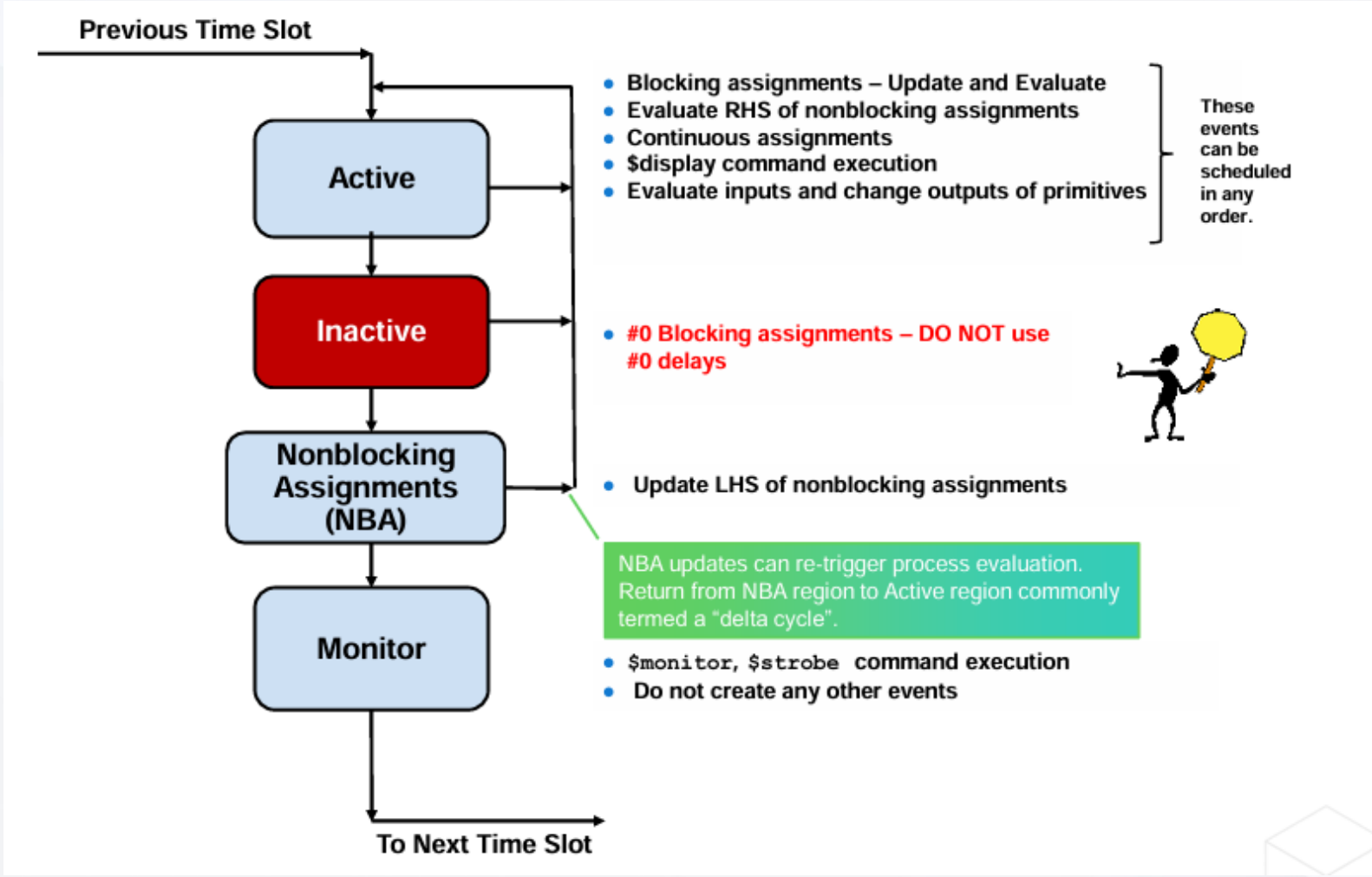
```
always @(posedge clk)
  begin
    d <=c;
    c <=b;
    b <=a;
  end
```





By: Mohammed Salah

Simulation Cycle



By: Mohammed Salah